

python

Introduction au langage

CFA INSTA

Institut National Supérieur
des Technologies Avancées

2025

Python : introduction

Sommaire

1. Présentation des participants
2. Historique de Python
3. Les avantages de Python
4. Notions de base
5. Types de variables
6. Opérations et formatage sur les chaînes
7. Conditions et opérateurs
8. Boucles et listes
9. Les fonctions
10. Classes
11. Modules et packages
12. Les types
13. Python avancé
14. Gestion des exceptions
15. Arguments des fonctions et fonctions anonymes
16. Compréhension de listes
17. Documentation
18. Opérations sur les fichiers

Organisation du cours

- Cours en miroir
- Classe inversée
- Partage des problèmes et solutions (Discord)
- Travaux pratiques
- QCM

Sources du cours

- https://realpython.com
- https://docs.python.org
- https://wikipedia.org
- https://www.w3schools.com
- https://www.docstring.fr/
- https://python.doctor

Présentations express

Qui êtes-vous ?

- Prénom - âge
- Poste actuel
- D’où je viens (études/job)
- Où je vais (idem)
- Centres d’intérêts (loisirs, sport, projets, vie associative…)
- Connaissances/expérience sur Python, la développement etc
- Attentes au sujet de ce cours

Python

Pourquoi s’intéresser à python ?

- ✓Développement web
- ✓Science des données
- ✓Analyse
- ✓Apprentissage automatique
- ✓Intelligence artificielle
- ✓Scripts et outils divers



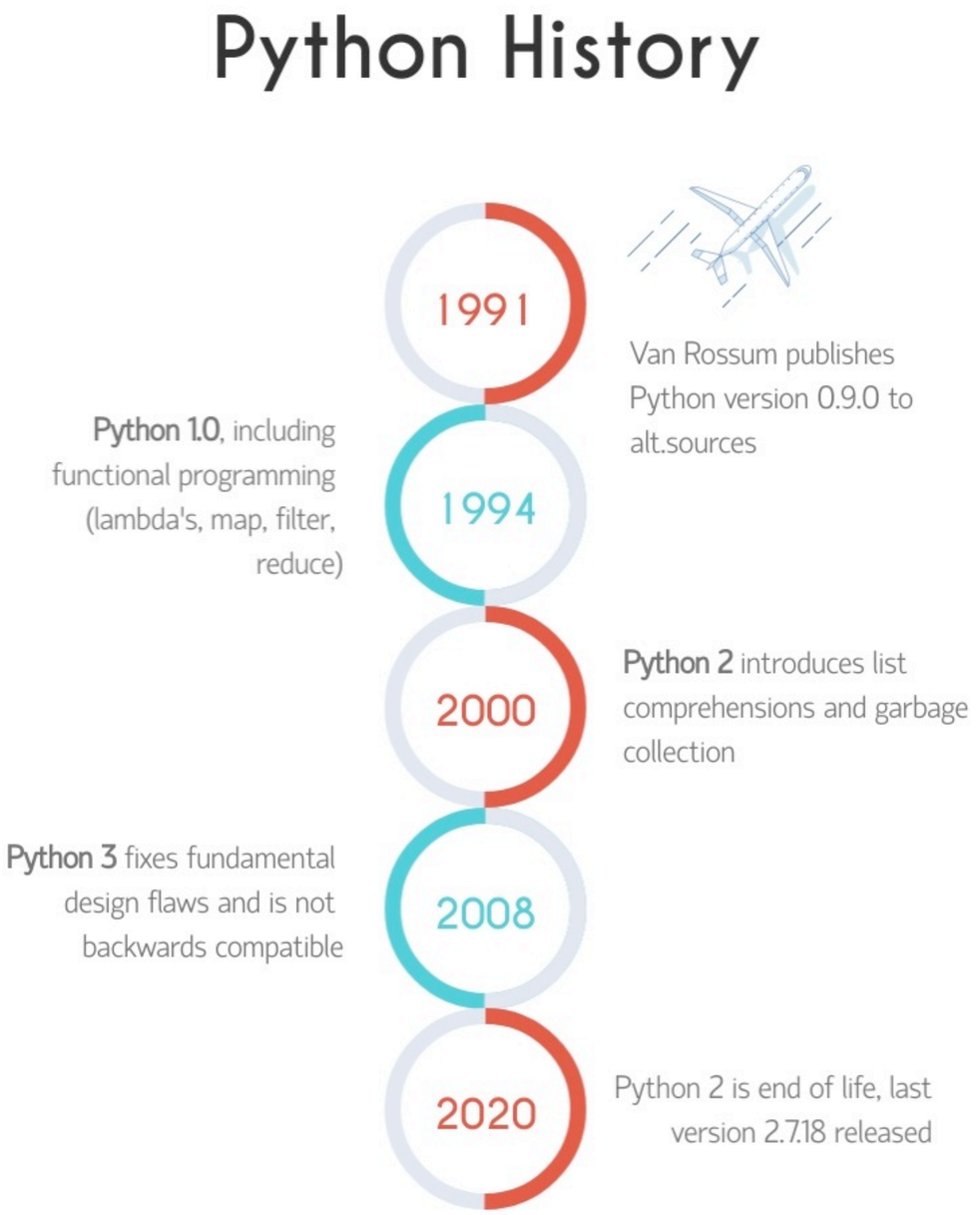
Python

Histoire

- Python a été créé par Guido van Gossum
- Le nom fait référence à « Monty Python »
- Première version publiée le 20 février 1991
- Python est facile à apprendre et il est conçu autour d'un ensemble de principes clairement définis (le *Zen de Python*) qui encourage le sprincipaux développeurs Python à créer un langage sans ambiguïté et facile à utiliser
- Les modules de Python sont écrits en Python purs (facilité) ou en C (performance)
- **Dernière version :** 3.14.0 (7 octobre 2025)



Guido Van Gossum



Python

Ecosystème





Pandas

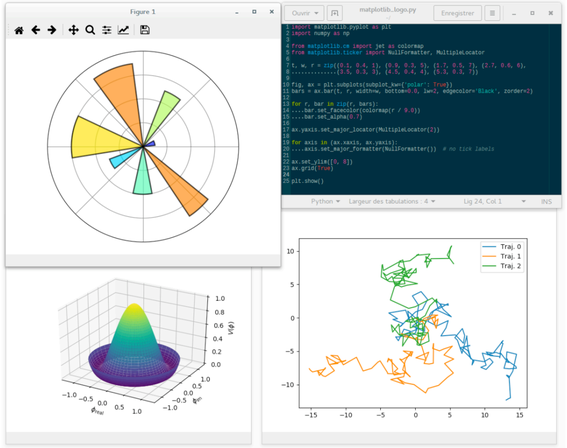




TensorFlow

- Bibliothèques graphiques
- Bibliothèques de traitement de données
- Bibliothèque d'IA et d'apprentissage automatique
- Bibliothèque de traitement de fichiers (comme Json, Yaml)
- Connexion de services externes
- Création de sites web (Django, FastApi, Flask)
- …

PySpark







TextBlob



Scrapy

Python

Installer Python

Sur MacOs : brew install python

Sur Linux : aptitude install python3

https://www.python.org/downloads/




```
➜  ~ python --version
Python 3.13.0
```

Python

Lancer python

Ouvrir un terminal :

- **Windows** : appuyez sur la touche windows et tapez *PowerShell*
- **MacOs** : Command+espace et tapez *terminal*
- **Linux** : Ctrl+Alt+T

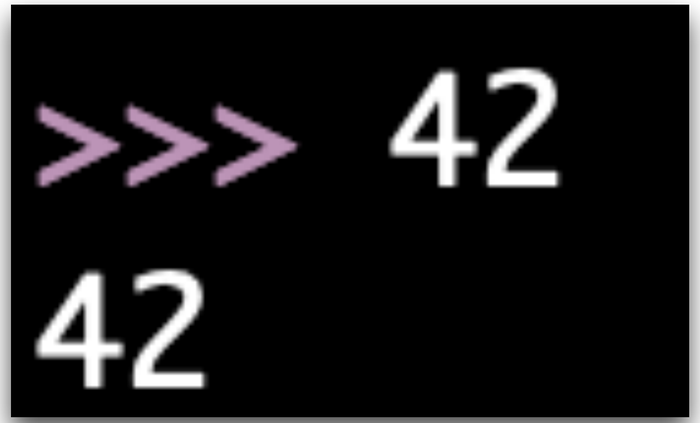
Executer Python : ***python3***

Quitter Python : **exit()** (ou alors Crl+d sous linux ou MacOs - sur windows :Ctrl+Z + Enter)

Python

REPL : Read-Eval Print Loop

Shell interactif qui vous permet de saisir des commandes Python et de voir directement le résultat : une bonne façon d'apprendre



- Read : Python lit 42
- Eval : Python évalue cette entrée et décide qu'il s'agit d'un nombre
- Print : Python imprime ce qui a été évalué
- Loop : Python est prêt pour l'entrée suivante

Python

Opérateurs arithmétiques

Python est un bon calculateur, vous pouvez utilisez REPL comme une calculatrice :



Python

Variables

```
x = 10
y = "Bonjour, Python!"
z = 3.14
```

- En programmation, **une variable** est **une référence à une valeur stockée en mémoire**, qui peut être **manipulée par le programme**.
- Imaginez une variable comme **une boîte étiquetée** dans laquelle vous pouvez **placer une valeur spécifique**.
- Chaque boîte a **un nom unique**, qui est utilisé pour **accéder à la valeur qu'elle contient**.
- En **Python**, **une variable est créée dès qu'une valeur lui est attribuée pour la première fois**, sans besoin de déclaration préalable.

Python

Types de variables

Python possède plusieurs types de variables

```
>>> type(result)
<class 'int'>
>>> sample_string = "Hello World"
>>> print(sample_string)
Hello World
>>> type(sample_string)
<class 'str'>
```

Python

Variables : plusieurs types

- Les entiers (Int)
- Les nombres à virgule flottante (float)
- Les nombres complexes (complex)
- Variables textuelles (str)
- Variables booléennes (bool)
- Variables de séquences (listes, tuples, chaines)

```
age = 30
temperature = -5

pi = 3.14159
poids = 70.5

nombre_complexe = 2+3j

salutation = "Bonjour"
nom = "Alice"

est_ete = True
est_vancance = False
```

Python

Variables : plusieurs types

```
fruits = ["pomme", "banane", "cerise"]


coordonnees = (10.0,20.0)


phrase = "Apprendre Python est amusant !"
```

- Variables de séquences (listes, tuples, chaines)
  - Listes (collection ordonnée de valeur modifiables)
  - Tuples (collection immuable)
  - Chaines (les chaines fonctionnent comme des listes de caractères

Python

Variables

Créer une variable **result**

```
>>> result
Traceback (most recent call last):
  File "<python-input-0>", line 1, in <module>
    result
NameError: name 'result' is not defined
```

```
>>> result = 3 * 5
>>> result
15
>>> result * 2
30
```

Python

Variables

```
>>> class = "cool"
  File "<stdin>", line 1
    class = "cool"
          ^
SyntaxError: invalid syntax
>>> coucou = "cool"
>>>
```

- Les noms de variables ne peuvent être composés que de **lettres (a-z, A-Z)**, **chiffres (0-9)** et **soulignements (_)**
- Les noms ne peuvent pas commencer par un chiffre
- Les noms sont sensibles à la casse
- Les développeurs pythons utilisent généralement le snake-case pour les variables et le camel-case pour les classes

Attention 

Les noms de variables ne peuvent pas être des mots clé réservés (class, if, then, else, while…)

Python

Les chaînes de caractères (Strings)

- Une chaine est une séquence de caractères
- Pour créer une chaine, il faut appliquer des guillemets
- Tout comme les nombres, les chaines peuvent être manipulées avec des opérateurs

```
>>> 'Hello Bob'
'Hello Bob'
>>> 'hello' + 'Insta'
'helloInsta'
>>> 'hello' * 4
'hellohellohellohello'
```

```
>>> "j'aime les pommes"
"j'aime les pommes"
>>> 'j\'aime les pommes'
"j'aime les pommes"
```

```
>>> """il s'agit d'une
... chaine sur
... plusieurs lignes"""
"il s'agit d'une\nchaine sur\nplusieurs lignes"
```

Python

Opérations sur les chaînes

```
>>> sampleString = "Hello Dude"
>>> sampleString.
```

- Créer une chaine
- Tapez le nom de cette chaine puis « . » puis appuyez sur TAB

```
>>> sampleString.upper()
'HELLO DUDE'
>>> len(sampleString)
10
>>> sampleString.split()
['Hello', 'Dude']
>>> sampleString[3]
'l'
>>> sampleString[::-1]
'eduD olleH'
```

Input	Method	Output
'hello WORLD'	.capitalize()	Hello world
'HELLO WORLD'	.lower()	hello world
'hello world'	.upper()	HELLO WORLD
'Python'	.center(10, '*')	**Python**
'HELLO WORLD'	.count('L')	3
'HELLO WORLD'	.index('O')	4
'HELLO WORLD'	.find('OR')	7
'31/01/2022'	.replace('/', '-')	31-01-2022
'31/01/2022'	.split('/')	['31', '01', '2022']
'abc123'	.isalnum()	True
'12345'	.isnumeric()	True
'hello world'	.islower()	True
'HELLO WORLD'	.isupper()	True

Python

Faire une première bonne impression

- print() est une fonction
- une fonction est quelque chose qu'on peut appeler et qui fera une action spécifique
- Les fonctions acceptent souvent des arguments
- La fonction print() permet d'imprimer du texte à l'écran

```
>>> print("Hi dude")
Hi dude
>>> print("Hi\t dude")
Hi      dude
>>> print("Hi\n dude")
Hi
 dude
```

Python

Formatage d'une chaîne et impression

```
>>> age = 42
>>> f'my age is {age}'
'my age is 42'
>>> name="guido"
>>> age="68"
>>> print(f'name={name}, age={age}')
name=guido, age=68
>>> print("this is my age:",42)
this is my age: 42
```

pour aller plus loin :

➡ https://docs.python.org/3/reference/lexical_analysis.html#f-strings

➡https://docs.python.org/3/tutorial/inputoutput.html#formatted-string-literals

Python

Programmation de boole et conditions



George Boole

Un **booléen** est le type de données le plus simple, il est soit à **True**, soit à **False**

```
>>> too_much_beer = True
>>> if too_much_beer:
...     print("You're gonna be drunk as usual")
...
You're gonna be drunk as usual
```

Attention à l'indentation !

Python

A condition d'avoir les bool

```
>>> too_much_beer = False
>>> if too_much_beer:
...     print("You're gonna be drunk as usual")
... else:
...     print("It's ok dude, you can drink again")
...
It's ok dude, you can drink again
```

nb : True vaut 1 et False faut 0

```
>>> True + 1
2
```

Opérateurs	Utilisation
>	Plus grand que
<	Plus petit que
>=	Plus grand ou égal
<=	Plus petit ou égal
==	égal
!=	non égal

Opérateurs	Exemple
and	True and False == False False and False == False True and True == True
or	True or False == True True or True == True False or False == False
not	not True == False not False == True

Python

Boucles et conditions

- 2 manières de créer des boucles en python : **for** et **while**
- Les boucles permettent de parcourir des **itérables**
- les **itérables** sont des objets qui peuvent renvoyer leurs membres un par un
- Tout est objet en python, et l'itérabilité est une propriété propre à certains objets
- Le modèle est :
  pour [variable] dans [itérable]:
  … action

```
>>> for letter in "Brooklyn":
...     print(letter)
...
B
r
o
o
k
l
y
n
```

Python

Les listes : un type propre à python

- Les listes sont un type de données en python
- Les listes peuvent contenir un ou plusieurs objets séparés par des virgules
- Les listes peuvent contenir plusieurs types d'objets

```
>>> myfirstlist = ["jake", "amy", 99]
>>> for thing in myfirstlist:
...     print(thing)
...
jake
amy
99
>>> print(myfirstlist[1])
amy
```

Python

Les listes : un type propre à python

✳Les listes peuvent être imbriquées !

```
>>> mysecondlist = ["Boyle", "Holt"]
>>> myfirstlist+mysecondlist
['jake', 'amy', 99, 'Boyle', 'Holt']
>>> mystrangelist = [33,["scully","hitchcock"]]
>>> mysecondlist+mystrangelist
['Boyle', 'Holt', 33, ['scully', 'hitchcock']]
>>> last_list = mysecondlist+mystrangelist
>>> last_list[3]
['scully', 'hitchcock']
```



Python

While : comment la boucler ?

- La boucle while est un peu plus simple que la boucle for
- elle suit la norme suivante :

  while [condition évaluée à true]:
  … action
- Essayez maintenant cette boucle sans l'incrémentation
- Tapez Ctrl+c pour couper l'exécution du programme

```
>>> i = 1
>>> while i < 4:
...         print(i)
...         i = i +1
...
1
2
3
```

Python

les fonctions

```
>>> value = len("coucou")
>>> value
6
>>> def sayHello():
...     print("Hello captain")
...
>>> sayHello()
Hello captain
```

- Une fonction est une section d'un programme qui exécute une tâche et peut renvoyer une valeur
- Une fonction est définie une fois et réutilisée plusieurs fois : ça sert à réutiliser le code
- Les fonctions acceptent des paramètres
- En python, les fonctions peuvent renvoyer plusieurs valeurs
- Il y a des fonction intégrées (print, len…)
- on peut définir soit même des fonction avec le mot clé **def**
- **attention à l'indentation !**

Python

les fonctions : argument choc

```
>>> sayHello("Pimento")
Hello Pimento
>>> def welcome(name,place):
...     print("Hi", name, "welcome to",place)
...
>>> welcome("Jake", "Brooklyn")
Hi Jake welcome to Brooklyn
```

- Les fonctions peuvent prendre un ou plusieurs arguments

Python

Return(jedi)

```
>>> def add(a, b):
...     return a + b
...
>>> add(19,23)
42
```

```
>>> if add(19,23) == 42:
...     print("that's it")
...
that's it
```

- Les fonctions peuvent retourner des trucs
- Les fonctions peuvent être utilisées dans des conditions

Python

Portée des variables

```
>>> def arrest():
...     print("Doug judy !")
...     var_doug = "arrested"
...
>>> arrest()
Doug judy !
>>> var_doug
Traceback (most recent call last):
  File "<python-input-52>", line 1, in <module>
    var_doug
NameError: name 'var_doug' is not defined
```

Python

fonctions : valeur par défaut

```
>>> def arrest(gangster = "Doug Judy"):
...     print("we arrested",gangster)
...
>>> arrest("Al Capone")
we arrested Al Capone
>>> arrest()
we arrested Doug Judy
```

Python

input

```
>>> def say_welcome():
...     name = input("enter your name please :")
...     print(name)
...
>>> say_welcome()
enter your name please :Harold Finch
Harold Finch
```

➡ **Exercice** : créez une fonction contenant une boucle infinie qui continue de demander le nom de l'utilisateur tout le temps

Python

input

➡Exemple de correction :

```
>>> def ask_again():
...     while True:
...         name = input("What's your name ?")
...         print("Hi",name)
...
>>> ask_again()
What's your name ?AMy
Hi AMy
What's your name ?Scully
Hi Scully
```

Python

IDE



PyCharm





Sublime Text

- Choisir un IDE
- Créer un projet et un premier fichier main.py
- Ajouter du code
- Exécuter avec la commande **python3 main.py**